

MANUAL DE REGLAS SINTÁCTICAS Y EJERCICIOS

Lenguaje NovaNyx 1.4.2 — Lenguajes y Autómatas 1

INSTITUTO TECNOLÓGICO DE NUEVO LAREDO

GRAMÁTICA OFICIAL IN01 - IN10

COMPILADOR 100% VERIFICADO

1. Nuevas Reglas Sintácticas Oficiales

A continuación se detallan las reglas añadidas y flexibilizadas en el analizador sintáctico (AnalizadorSintactico.cs) para apegarse a los diagramas sintácticos oficiales del lenguaje y garantizar una escritura robusta:

1.1. Uso del Delimitador ';' (DEL / SC;)

Instrucción / Estructura	Diagrama Oficial	Delimitador ';'	Ejemplo Válido
Inicio de Programa	IN01 (START DEL)	Opcional / Permitido	START ; o START; o START
Fin de Programa	IN02 (END DEL)	Opcional / Permitido	END ; o END; o END
Asignación	ASIGN (IDV = ARG4 DEL)	Obligatorio	x = 10 ; o x = 10;
Lectura (READ)	IN04 (READ ARG4 DEL)	Obligatorio	READ a , b ;
Escritura (PRINT)	IN03 (PRINT ARG3 DEL)	Obligatorio	PRINT "Total: " , suma ;
Cierre de Bloques (ENDIF, ENDCASE, ENDWHILE, ENDFOR, ENDDO)	IN05_3, IN06_3, IN07_1, IN08_1, IN10_1	Opcional / Permitido	ENDWHILE ; o ENDWHILE;
Bifurcaciones (THEN, ELSE)	IN05_1, IN05_2	Opcional / Permitido	THEN o THEN; / ELSE o ELSE;
Etiquetas de Caso (CASE, NONE)	IN06_1, IN06_2	Admite ';' o ':'	CASE 1 ; o CASE 1: / NONE ; o NONE:

🌟 **Delimitadores Pegados o con Espacio:** Gracias al normalizador léxico de Form1.cs, el código puede escribirse tanto con espacio antes del delimitador (x = 10 ;) como pegado (x = 10;, START;, ENDCASE;).

1.2. Reglas de Estructura por Instrucción

- **WHILE (Diagrama IN10):** La palabra DO no es obligatoria. Se permite tanto WHILE condicion ... ENDWHILE; como la variante con DO: WHILE condicion DO ... ENDWHILE;.

- **PERHAPS (Diagrama IN06):** Acepta evaluación de variable o expresión. Cada bloque CASE y NONE puede terminar con punto y coma (;) o dos puntos (:). El cierre del bloque siempre se realiza con ENDCASE;.
- **FOR (Diagrama IN07):** Admite la sintaxis estricta del diagrama: FOR FROM idx = 1 UNTIL 10 INTERVAL 1 o con condición explícita UNTIL idx <= 10. Se cierra con ENDFOR;.
- **DO WHILE (Diagrama IN08):** Admite la sintaxis DO WHILE condicion ... ENDDO; o el bloque DO ... WHILE condicion ENDDO;.
- **EXECUTE (Diagrama IN09):** Ejecuta el cuerpo de instrucciones hasta cumplir la condición: EXECUTE ... UNTIL condicion ;.
- **Cadenas de Texto (STR):** Como el autómata de la base de datos no contiene el carácter de espacio en el alfabeto, las frases deben escribirse con guion bajo (ej. "Mayor_de_edad", "Total_Pagar") o palabras simples ("Uno", "Aprobado").

2. Ejercicios de Prueba Válidos (1 de cada Estructura)

Todos los siguientes ejercicios han sido probados y validados con **0 errores** contra la base de datos SQL:

Ejercicio 1: Asignaciones y Operaciones Aritméticas

0 ERRORES

Verifica START ;, END ;, asignaciones y expresiones con paréntesis y jerarquía.

```
START ;  
x = 10 ;  
y = 20 ;  
resultado = ( x + y ) * 2 ;  
END ;
```

Ejercicio 2: Entrada y Salida (READ y PRINT con Múltiples Argumentos)

0 ERRORES

Verifica lectura múltiple con comas (IN04) e impresión combinada de texto y variable (IN03).

```
START ;  
READ val1 , val2 ;  
suma = val1 + val2 ;  
PRINT "La_suma_es" , suma ;  
END ;
```

Ejercicio 3: Decisión IF / THEN / ELSE / ENDIF (Diagrama IN05)

0 ERRORES

Verifica condición relacional, rama verdadera (THEN), rama falsa (ELSE) y cierre ENDIF.

```
START ;  
edad = 18 ;  
IF edad >= 18 THEN  
    PRINT "Mayor" ;  
ELSE  
    PRINT "Menor" ;  
ENDIF ;  
END ;
```

Ejercicio 4: Selección Múltiple PERHAPS / CASE / NONE / ENDCASE (Diagrama IN06)

0 ERRORES

Verifica casos con ';' o ':', bloque por defecto (NONE) y cierre correcto ENDCASE.

```
START ;
opcion = 2 ;
PERHAPS opcion ;
CASE 1 ;
    PRINT "Uno" ;
CASE 2 ;
    PRINT "Dos" ;
NONE ;
    PRINT "Otro" ;
ENDCASE ;
END ;
```

Ejercicio 5: Ciclo WHILE Estándar (Diagrama IN10 sin DO)

0 ERRORES

Verifica condición lógica previa y cierre ENDWHILE sin exigir la palabra reservada DO.

```
START ;
contador = 1 ;
WHILE contador <= 5
    PRINT contador ;
    contador = contador + 1 ;
ENDWHILE ;
END ;
```

Ejercicio 6: Ciclo WHILE con palabra DO Opcional

0 ERRORES

Verifica compatibilidad cuando el usuario o alumno decide incluir la palabra DO.

```
START ;
contador = 1 ;
WHILE contador <= 5 DO
    PRINT contador ;
    contador = contador + 1 ;
ENDWHILE ;
END ;
```

Ejercicio 7: Ciclo FOR (Diagrama IN07)

0 ERRORES

Verifica sintaxis oficial FOR FROM id = inicio UNTIL fin INTERVAL paso y cierre ENDFOR.

```
START ;
FOR FROM idx = 1 UNTIL 10 INTERVAL 1
    PRINT idx ;
ENDFOR ;
END ;
```

Ejercicio 8: Ciclo DO WHILE (Diagrama IN08)

0 ERRORES

Verifica cabecera conjunta DO WHILE condicion y cierre ENDDO.

```
START ;
num = 0 ;
DO WHILE num < 3
    num = num + 1 ;
    PRINT num ;
ENDDO ;
END ;
```

Ejercicio 9: Ciclo EXECUTE (Diagrama IN09)

0 ERRORES

Verifica cuerpo de ejecución directa y validación final con UNTIL condicion ;.

```
START ;
total = 1 ;
EXECUTE
    total = total * 2 ;
    PRINT total ;
UNTIL total >= 16 ;
END ;
```

Ejercicio 10: Condición Compuesta con Paréntesis y Operador Lógico AND

0 ERRORES

Verifica subcondiciones agrupadas y el operador lógico oficial AND (LO1).

```
START ;
nota = 85 ;
asistencia = 90 ;
IF ( nota >= 70 ) AND ( asistencia >= 80 ) THEN
    PRINT "Aprobado" ;
ENDIF ;
END ;
```

3. Ejercicios de Prueba para Comprobar la Detección de Errores

Utiliza estos ejercicios para verificar que la tabla dgtErrores resalta en color anaranjado y rojo los errores sintácticos:

Prueba de Error 1: Olvido de END al final

DETECCIÓN DE ESTRUCTURA

```
START ;
x = 5 ;
```

Mensaje esperado en tabla: "Error Sintáctico: Se esperaba el fin del programa (END / RW02)."

Prueba de Error 2: Falta de Delimitador ';' en Asignación

DETECCIÓN DE DELIMITADOR

```
START ;  
x = 10  
y = 20 ;  
END ;
```

Mensaje esperado en tabla: "Error Sintáctico: Se esperaba delimitador ';' (DEL) al final de la asignación."

Prueba de Error 3: Condición Inválida sin Operador de Comparación

DETECCIÓN LÓGICA

```
START ;  
IF x + 5 THEN  
    PRINT "Invalido" ;  
ENDIF ;  
END ;
```

Mensaje esperado en tabla: "Error Sintáctico: Se esperaba un operador relacional (>, <, ==, !=, >=, <=) o lógico en la condición."